



Progress Report 5

BC Personal Health Management Platform

An AI-Powered Healthcare Navigation and Analytics System

CSIS 4495 - 002: Applied Research Project
Winter 2026

Prepared For:

Padmapriya Arasanipalai Kandhadai

Date: March 23, 2026

Github Link: https://github.com/desporamon/W26_4495_S2_DesmondC

Prepared By:

<u>Name</u>	<u>Student ID</u>	<u>Email Address</u>
Desmond Chua	300369803	chuad1@student.douglascollege.ca

Contents

1. Work Date/Hours Log	2
2. Summary Description of Work Done.....	6
2.1 Sprint 5A Overview: Scope Decision and Testing Strategy	6
2.2 Sprint 5A.1: Tests Folder Structure and conftest.py	6
2.3 Sprint 5A.2: Authentication Unit Tests (test_auth.py — 10 tests)	7
2.4 Sprint 5A.3: CTAS Safety Rules Unit Tests (test_ctas_rules.py — 23 tests).....	8
2.5 Sprint 5A.4: Database Unit Tests (test_database.py — 12 tests).....	8
2.6 Sprint 5A.5: ML Model Unit Tests (test_ml_model.py — 12 tests).....	9
2.7 Sprint 5A.6: Pipeline Integration Tests (test_pipeline.py — 6 tests).....	10
2.8 Sprint 5A.7: Clinical Scenario Integration Tests (test_chatbot_scenarios.py — 6 tests).....	10
2.9 Sprint 5A.8: Dashboard Headless UI Tests (test_dashboard_apptest.py — 8 tests).....	11
2.10 Sprint 5A.9: Chatbot Headless UI Tests (test_chatbot_apptest.py — 8 tests).....	12
2.11 Sprint 5A.10: Property-Based Fuzz Tests (test_fuzz_hypothesis.py — 10 tests).....	12
2.12 Sprint 5A.11: Full Test Suite Run.....	13
2.13 Issues Encountered and Solutions	14
3. Repo Check-in of Implementation Completed.....	17
Git Commit History (this reporting period).....	19
4. Revised Timeline and Next Steps.....	21
4.1 What's Next: Sprint 5B, 5C and Sprint 6.....	22
4.2 Scope Decision: Community Pulse Formally Descoped	22
4.3 QA Perspective Testing — Post-Submission Portfolio	22
5. AI Use Section.....	23
5.1 AI Tools Usage Summary.....	23
5.2 Table of AI Tools and Specific Use	23
5.3 Statement of Originality	26
Appendix A: AI Prompt History.....	27
Prompt Session 1: Understanding pytest Fixture Patterns for conftest.py Design	27
Prompt Session 2: Mocking Streamlit Dependencies for Chatbot Module Testing	27
Prompt Session 3: Property-Based Testing with Hypothesis for Healthcare Validation Functions .	28
Prompt Session 4: Streamlit AppTest Framework for Headless UI Testing.....	29

1. Work Date/Hours Log

Note: Logs are entered as work is completed with granular entries. This reporting period covers Sprint 5A (Automated Developer Testing Suite) from March 17–23, 2026. The Community Pulse feature was formally descoped this period in favor of a comprehensive testing implementation (see Section 4.1 for rationale).

Date	Hours	Description of Work Done
2026-03-17	1.00	Sprint 5A scope planning: Decided to drop Community Pulse (web scraping) feature and focus on automated testing suite. Drafted scope decision rationale and sent email to Prof. Priya explaining the scope change and requesting absence from class for job interview
2026-03-17	0.75	Sprint 5A.1: Designed tests/ folder architecture — unit/, integration/, ui/, property/ sub-packages. Created all <code>_init_.py</code> files, <code>conftest.py</code> with <code>test_db</code> (in-memory SQLite fixture with yield teardown) and <code>test_user</code> fixtures, <code>TEST_REGISTER.md</code> with table header
2026-03-17	0.50	Sprint 5A.1 cleanup: Identified and removed old ad-hoc test scripts (<code>test_openai.py</code> , <code>test_predict.py</code> , <code>test_rules.py</code>) from <code>Implementation/tests/</code> — superseded by new proper pytest suite at root <code>tests/</code> folder
2026-03-19	1.00	Sprint 5A.2: Read <code>auth.py</code> source code to understand exact function signatures (<code>hash_password</code> , <code>verify_password</code> , <code>validate_password_strength</code> , <code>validate_email</code> , <code>require_authentication</code>). Designed 10 test cases across 4 logical groups: Registration, Login, Security, Session
2026-03-19	1.25	Sprint 5A.2: Wrote <code>test_auth.py</code> — 10 unit tests for authentication module. Used <code>conftest.py test_db</code> fixture for all database operations, <code>unittest.mock</code> for Streamlit <code>session_state</code> and <code>st.switch_page</code> . Verified all 10 tests pass: 10/10 passed in 4.38s
2026-03-19	0.50	Sprint 5A.3 analysis: Read <code>rules.py</code> to confirm <code>check_critical_symptoms()</code> signature, return dict structure, 14-rule count, match types (any vs all), case-insensitive matching behaviour before writing tests

Date	Hours	Description of Work Done
2026-03-21	1.50	Sprint 5A.3: Wrote test_ctas_rules.py — 23 unit tests across 3 groups: 14 parametrized positive detection tests (one per CTAS rule), 5 negative pass-through tests, 4 edge case tests (mixed case, None input, large lists, single keyword). All 23 passing
2026-03-21	0.75	Sprint 5A.4 analysis: Read database.py (690 lines) — confirmed all CRUD function signatures, return types, table schema, and connection pattern (get_connection() called internally — requires unittest.mock.patch to redirect to test DB)
2026-03-21	1.25	Sprint 5A.4: Wrote test_database.py — 12 unit tests across 3 groups: 5 User CRUD tests, 5 Assessment CRUD tests, 2 Data Integrity tests. Patched get_connection() with test DB factory. Verified user isolation (TC-DB-11) and correct FK constraint handling. All 12 passing
2026-03-21	0.75	Sprint 5A.5 analysis: Confirmed ML layer lives in chatbot.py (not ml_model.py). Read model.pkl and model_metadata.json directly — identified feature vector building logic, 70% confidence threshold location, predict_proba() call pattern
2026-03-21	1.25	Sprint 5A.5: Wrote test_ml_model.py — 12 unit tests across 4 groups: model loading (2), confidence threshold (3), input handling (4), output validation (3). Loaded real model.pkl directly without Streamlit. UTI symptoms achieved 100% confidence. All 12 passing
2026-03-22	0.75	Sprint 5A.6 analysis: Read chatbot.py — confirmed run_assessment() layer sequence (OpenAI → CTAS → ML), all 4 return statuses and their dict keys, patch target (components.chatbot.extract_symptoms — patch where used not where defined)
2026-03-22	1.25	Sprint 5A.6: Wrote test_pipeline.py — 6 integration tests. Mocked both extract_symptoms() and get_openai_fallback_guidance(). Tested emergency

Date	Hours	Description of Work Done
		circuit breaker, ML prediction path, OpenAI fallback path, safety invariants. All 6 passing in 3.74s
2026-03-23	0.75	Sprint 5A.7 analysis: Confirmed <code>extract_symptoms()</code> return dict key is 'symptoms' (not 'extracted_symptoms'). Confirmed all mock return structures for 6 clinical scenarios before writing tests
2026-03-23	1.25	Sprint 5A.7: Wrote <code>test_chatbot_scenarios.py</code> — 6 integration tests covering cardiac emergency, stroke emergency, UTI ML prediction, vague symptom fallback, emergency override safety, minimal input edge case. All 6 passing on first run
2026-03-23	0.75	Sprint 5A.8 analysis: Read <code>3_My_Dashboard.py</code> — confirmed <code>session_state</code> keys, data source (<code>get_user_assessments</code> via <code>components.database</code>), 4 stat cards as custom HTML <code>st.markdown</code> (not <code>st.metric</code>), 2 selectboxes, CWD dependency for <code>assets/logo.png</code>
2026-03-23	1.50	Sprint 5A.8: Wrote <code>test_dashboard_apptest.py</code> — 8 headless UI tests using Streamlit AppTest. Implemented <code>_run_dashboard()</code> helper with <code>os.chdir(SRC_DIR)</code> pattern. Verified stat card headers, selectbox options (All Time, Last 3 Months, All Urgency Levels), user data isolation. All 8 passing
2026-03-23	0.75	Sprint 5A.9 analysis: Read <code>2_Symptom_Assessment.py</code> — confirmed <code>session_state</code> keys, <code>process_multiturn_message()</code> as primary function to mock (not <code>run_assessment()</code>), AppTest chat input syntax (<code>at.chat_input[0].set_value().run()</code>), <code>_submit_chat()</code> helper pattern needed
2026-03-23	1.50	Sprint 5A.9: Wrote <code>test_chatbot_apptest.py</code> — 8 headless UI tests. Implemented <code>_run_chatbot()</code> and <code>_submit_chat()</code> helpers with patch re-application on rerun. Tested warning banner, welcome message personalisation, ML response, emergency response, auth guard. All 8 passing

Date	Hours	Description of Work Done
2026-03-23	0.75	Sprint 5A.10 analysis: Confirmed Hypothesis installed (6.151.9). Read auth.py and rules.py to verify exact validation rules — password requires letter + digit (not uppercase + lowercase separately). Corrected strategy before writing
2026-03-23	1.25	Sprint 5A.10: Wrote test_fuzz_hypothesis.py — 10 property-based tests using Hypothesis. 1,000 total fuzz inputs (100 per test). Covered password fuzzing (3 tests), email fuzzing (3 tests), CTAS rules fuzzing (3 tests), database fuzzing (1 test). All 10 passing
2026-03-23	0.50	Sprint 5A.11: Ran full test suite from project root — 79 tests across unit, integration, property folders all passing in 20.89s. UI tests confirmed passing individually (16 tests). Took screenshots for progress report documentation
2026-03-23	0.25	Git commits: All Sprint 5A tasks committed with granular messages (5A.1 through 5A.11) and pushed to GitHub main branch

TOTAL 22.25 Hours	Hours logged for Progress Report 5 (Mar 17–23, 2026)
--------------------------	---

2. Summary Description of Work Done

During this reporting period (March 17- 23, 2026), the main focus changed from creating new features to ensuring the quality of the product. As a result of Sprint 5A, a full automated test suite was created that covers every part of the BC Health Platform code. This was a conscious scope choice: instead of developing the Community Pulse feature through web scraping, the developer's time was devoted to creating a multi-layer testing pipeline that reveals a full software development lifecycle design build test, and deploy. 95 automated test cases were made and carried out in four testing types: unit, integration, UI (headless), and property-based fuzz testing.

2.1 Sprint 5A Overview: Scope Decision and Testing Strategy

A formal scope decision was carried out to omit the Community Pulse (Reddit web scraping) feature and substitute it with a thorough automated testing suite. The justification for this decision was threefold. Firstly, scraping of healthcare authority websites involves raising data governance issues which are not proper for a proof-of-concept healthcare platform without explicit permission agreements. Secondly, the platform was almost done with the main features after Sprint 4B so a full software quality assurance layer would be a better academic contribution than one more data feature. Thirdly, as a student and a QA role seeker in parallel with data analytics, the chance to create automated testing tools aboard was direct career relevant experience.

The testing plan that we used is based on industry standard shift-left testing principles according to which testing is done not only after the completion of development but also through its course. As a result, the test suite consists of four tiers, testing different facets of the system.

2.2 Sprint 5A.1: Tests Folder Structure and conftest.py

The very first step laid down the formal testing framework for the whole effort. A tests/ folder was established right at the top-level of the project (i. e. at the outside of Implementation/) which housed four sub-packages: unit/, integration/, ui/, and property/. This layout is a reflection of industry standard pytest project files organization and the separation of different types of tests based on their scope and purposes.

A critical component of Sprint 5A.1 was conftest.py — the shared pytest fixture file. Understanding what conftest.py does is essential to understanding how all subsequent tests work. pytest automatically loads conftest.py before running any test in the same directory or sub-directories. It contains two shared fixtures:

- test_db: Creates a fresh temporary SQLite database in memory (:memory:) using sqlite3.connect. The keyword :memory: means the database exists entirely in RAM — it is

instant to create, completely empty each time, and destroyed automatically when the connection closes. The real health_platform.db is never touched. The yield keyword splits the fixture into setup (before yield — creates the database) and teardown (after yield — closes the connection). Every test that uses this fixture receives its own isolated database.

- test_user: Provides a standard test user dictionary (name, email, password, age, gender, health_region) for use by any test that needs to simulate user registration or login.

Also, three old ad-hoc test scripts (test_openai.py, test_predict.py, test_rules.py) dating from the very first development sprints were deleted from Implementation/tests/ as these were simple manual checking scripts now replaced by the formal pytest suite.

2.3 Sprint 5A.2: Authentication Unit Tests (test_auth.py — 10 tests)

Ten unit tests were developed for components/auth.py and segregated into four logical groups: Registration (TC-AUTH-01 to TC-AUTH-04), Login (TC-AUTH-05 to TC-AUTH-07), Security (TC-AUTH-08), and Session (TC-AUTH-09 to TC-AUTH-10). All the db operations are done through the test_db fixture, so the real health_platform.db file is never touched. Streamlit calls (st.session_state, st.stop, st.switch_page) are mocked with unittest.mock to make tests work without an actual Streamlit instance.

Before test writing, the test file looks at the auth.py source code and it uses the exact function signatures for the test design (hash_password verify_password validate_password_strength validate_email require_authentication). Besides a clear docstring, each test has descriptive assertion failure messages as well. As a security measure, TC-AUTH-08 checks bcrypt storage by asserting that the stored hash begins with \$2b\$ and the stored hash and the plain text password are not equal.

```

• (venv) PS C:\Users\user\OneDrive\Desmond_New\BC-Health-Platform> python -m pytest tests/unit/test_auth.py -v
===== test session starts =====
platform win32 -- Python 3.13.5, pytest-9.0.2, pluggy-1.6.0 -- C:\Users\user\OneDrive\Desmond_New\BC-Health-Platform\venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\user\OneDrive\Desmond_New\BC-Health-Platform
plugins: anyio-4.12.1
collected 10 items

tests/unit/test_auth.py::TestRegistration::test_tc_auth_01_valid_registration PASSED [ 10%]
tests/unit/test_auth.py::TestRegistration::test_tc_auth_02_duplicate_email PASSED [ 20%]
tests/unit/test_auth.py::TestRegistration::test_tc_auth_03_weak_password PASSED [ 30%]
tests/unit/test_auth.py::TestRegistration::test_tc_auth_04_invalid_email_format PASSED [ 40%]
tests/unit/test_auth.py::TestLogin::test_tc_auth_05_valid_login PASSED [ 50%]
tests/unit/test_auth.py::TestLogin::test_tc_auth_06_wrong_password PASSED [ 60%]
tests/unit/test_auth.py::TestLogin::test_tc_auth_07_nonexistent_email PASSED [ 70%]
tests/unit/test_auth.py::TestSecurity::test_tc_auth_08_password_stored_as_bcrypt_hash PASSED [ 80%]
tests/unit/test_auth.py::TestSession::test_tc_auth_09_logout_clears_session PASSED [ 90%]
tests/unit/test_auth.py::TestSession::test_tc_auth_10_page_protection_redirects PASSED [100%]

===== 10 passed in 3.61s =====

```

Fig 2.3 - pytest_test_auth_results.png — 10 passed in 3.61s

2.4 Sprint 5A.3: CTAS Safety Rules Unit Tests (test_ctas_rules.py — 23 tests)

This is the most safety-critical test file in the entire suite. The CTAS (Canadian Triage and Acuity Scale) rules form the Emergency Circuit Breaker layer — if these rules fail, patients could receive incorrect urgency guidance. Every single one of the 14 rules defined in rules.py must be verified.

The 23 tests are organized into three groups. Part 1 uses `pytest.mark.parametrize` to dynamically build 14 positive detection tests from the actual `CRITICAL_RULES` list in rules.py — one test per rule, automatically picking up any future rule additions without code changes. Part 2 has 5 negative pass-through tests confirming that common non-emergency symptoms (sore throat, headache, fever alone) correctly return `is_critical=False`. Part 3 has 4 edge case tests covering mixed-case input, single exact keyword matching, large non-emergency symptom lists, and `None` input.

The `parametrize` approach is particularly important for the three rules using `match_type='all'` (Cardiac Arrest, Stroke, Suspected Meningitis) — the test inputs for these rules include all required keywords, not just one, since every keyword must be present to trigger the rule.

```
(venv) PS C:\Users\user\OneDrive\Desmond_New\BC-Health-Platform> python -m pytest tests/unit/test_ctas_rules.py -v
===== test session starts =====
platform win32 -- Python 3.13.5, pytest-9.0.2, pluggy-1.6.0 -- C:\Users\user\OneDrive\Desmond_New\BC-Health-Platform\venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\user\OneDrive\Desmond_New\BC-Health-Platform
plugins: anyio-4.12.1
collected 23 items

tests/unit/test_ctas_rules.py::TestPositiveDetection::test_rule_triggers_correctly[Cardiac Arrest / Heart Attack] PASSED [ 4%]
tests/unit/test_ctas_rules.py::TestPositiveDetection::test_rule_triggers_correctly[Stroke (FAST Signs)] PASSED [ 8%]
tests/unit/test_ctas_rules.py::TestPositiveDetection::test_rule_triggers_correctly[Unconsciousness / Unresponsive] PASSED [ 13%]
tests/unit/test_ctas_rules.py::TestPositiveDetection::test_rule_triggers_correctly[Severe Airway Obstruction] PASSED [ 17%]
tests/unit/test_ctas_rules.py::TestPositiveDetection::test_rule_triggers_correctly[Poisoning / Overdose] PASSED [ 30%]
tests/unit/test_ctas_rules.py::TestPositiveDetection::test_rule_triggers_correctly[Difficulty Breathing] PASSED [ 34%]
tests/unit/test_ctas_rules.py::TestPositiveDetection::test_rule_triggers_correctly[Severe Allergic Reaction / Anaphylaxis] PASSED [ 39%]
tests/unit/test_ctas_rules.py::TestPositiveDetection::test_rule_triggers_correctly[Chest Pain] PASSED [ 43%]
tests/unit/test_ctas_rules.py::TestPositiveDetection::test_rule_triggers_correctly[Severe Head Injury] PASSED [ 47%]
tests/unit/test_ctas_rules.py::TestPositiveDetection::test_rule_triggers_correctly[Suspected Meningitis] PASSED [ 52%]
tests/unit/test_ctas_rules.py::TestPositiveDetection::test_rule_triggers_correctly[Severe Burns] PASSED [ 56%]
tests/unit/test_ctas_rules.py::TestPositiveDetection::test_rule_triggers_correctly[Mental Health Crisis \u2014 Suicidal Thoughts / Self-Harm] PASSED [ 60%]
tests/unit/test_ctas_rules.py::TestNegativePassthrough::test_nc01_sore_throat PASSED [ 65%]
tests/unit/test_ctas_rules.py::TestNegativePassthrough::test_nc02_headache_fatigue PASSED [ 69%]
tests/unit/test_ctas_rules.py::TestNegativePassthrough::test_nc03_runny_nose_mild_cough PASSED [ 73%]
tests/unit/test_ctas_rules.py::TestNegativePassthrough::test_nc04_empty_list PASSED [ 78%]
tests/unit/test_ctas_rules.py::TestNegativePassthrough::test_nc05_fever_alone PASSED [ 82%]
tests/unit/test_ctas_rules.py::TestEdgeCases::test_ec01_single_exact_keyword PASSED [ 86%]
tests/unit/test_ctas_rules.py::TestEdgeCases::test_ec02_mixed_case_input PASSED [ 91%]
tests/unit/test_ctas_rules.py::TestEdgeCases::test_ec03_large_non_emergency_list PASSED [ 95%]
tests/unit/test_ctas_rules.py::TestEdgeCases::test_ec04_none_input_does_not_crash PASSED [100%]

===== 23 passed in 0.18s =====
```

Fig 2.4 - `pytest_test_ctas_rules_results.png` — 23 passed in 0.18s

2.5 Sprint 5A.4: Database Unit Tests (test_database.py — 12 tests)

12 tests across three groups: User CRUD (5), Assessment CRUD (5), and Data Integrity (2). The key technical challenge was that `database.py` creates its own SQLite connection internally via `get_connection()` — it does not accept a connection parameter. To keep tests isolated from the real `health_platform.db`, `unittest.mock.patch` was used to redirect `get_connection()` to the temporary test database instead.

Two tests worth highlighting: TC-DB-11 verifies user data isolation — User A's assessments are never visible to User B, which is a core healthcare privacy requirement. TC-DB-04 asserts a None return value (not an exception) for duplicate email registration, because create_user() handles the IntegrityError internally — a behaviour confirmed by reading the source code first.

```
(venv) PS C:\Users\user\OneDrive\Desmond_New\BC-Health-Platform> python -m pytest tests/unit/test_database.py -v
cachedir: .pytest_cache
rootdir: C:\Users\user\OneDrive\Desmond_New\BC-Health-Platform
plugins: anyio-4.12.1
collected 12 items

tests/unit/test_database.py::TestUserCRUD::test_tc_db_01_create_user_valid PASSED [ 8%]
tests/unit/test_database.py::TestUserCRUD::test_tc_db_02_get_user_by_email_existing PASSED [ 16%]
tests/unit/test_database.py::TestUserCRUD::test_tc_db_03_get_user_by_email_nonexistent PASSED [ 25%]
tests/unit/test_database.py::TestUserCRUD::test_tc_db_04_duplicate_email_returns_none PASSED [ 33%]
tests/unit/test_database.py::TestUserCRUD::test_tc_db_05_password_stored_as_bcrypt_hash PASSED [ 41%]
tests/unit/test_database.py::TestAssessmentCRUD::test_tc_db_06_save_assessment_valid PASSED [ 50%]
tests/unit/test_database.py::TestAssessmentCRUD::test_tc_db_07_get_assessments_by_user_id PASSED [ 58%]
tests/unit/test_database.py::TestAssessmentCRUD::test_tc_db_08_get_assessments_empty_list PASSED [ 66%]
tests/unit/test_database.py::TestAssessmentCRUD::test_tc_db_09_assessment_stores_all_fields PASSED [ 75%]
tests/unit/test_database.py::TestAssessmentCRUD::test_tc_db_10_save_assessment_invalid_user_id PASSED [ 83%]
tests/unit/test_database.py::TestDataIntegrity::test_tc_db_11_user_isolation PASSED [ 91%]
tests/unit/test_database.py::TestDataIntegrity::test_tc_db_12_database_initialises_tables PASSED [100%]

===== 12 passed in 0.49s =====
```

Fig 2.5 - pytest_test_database_results.png — 12 passed in 0.49s

2.6 Sprint 5A.5: ML Model Unit Tests (test_ml_model.py — 12 tests)

A key finding during analysis: there is no standalone ml_model.py — the ML layer lives inside chatbot.py, which cannot be imported directly in a test environment due to Streamlit dependencies. The solution was to load model.pkl and model_metadata.json directly using joblib and json, bypassing chatbot.py entirely. The feature vector building replicates chatbot.py's normalization logic exactly: symptom.lower().strip().replace(' ', '_'), matched against the 131-entry symptom_list, with severity weights applied. TC-ML-03 uses UTI symptoms to confirm 100% confidence. TC-ML-05 tests the 70% threshold as a direct logic assertion rather than depending on a specific model output value.

```
(venv) PS C:\Users\user\OneDrive\Desmond_New\BC-Health-Platform> python -m pytest tests/unit/test_ml_model.py -v
===== test session starts =====
platform win32 -- Python 3.13.5, pytest-9.0.2, pluggy-1.6.0 -- C:\Users\user\OneDrive\Desmond_New\BC-Health-Platform\env\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\user\OneDrive\Desmond_New\BC-Health-Platform
plugins: anyio-4.12.1
collected 12 items

tests/unit/test_ml_model.py::TestModelLoading::test_tc_ml_01_model_loads_successfully PASSED [ 8%]
tests/unit/test_ml_model.py::TestModelLoading::test_tc_ml_02_metadata_is_complete PASSED [ 16%]
tests/unit/test_ml_model.py::TestConfidenceThreshold::test_tc_ml_03_high_confidence_prediction PASSED [ 25%]
tests/unit/test_ml_model.py::TestConfidenceThreshold::test_tc_ml_04_low_confidence_prediction PASSED [ 33%]
tests/unit/test_ml_model.py::TestConfidenceThreshold::test_tc_ml_05_confidence_boundary_logic PASSED [ 41%]
tests/unit/test_ml_model.py::TestInputHandling::test_tc_ml_06_valid_symptom_list PASSED [ 50%]
tests/unit/test_ml_model.py::TestInputHandling::test_tc_ml_07_empty_symptom_list PASSED [ 58%]
tests/unit/test_ml_model.py::TestInputHandling::test_tc_ml_08_single_symptom_input PASSED [ 66%]
tests/unit/test_ml_model.py::TestInputHandling::test_tc_ml_09_unknown_symptoms PASSED [ 75%]
tests/unit/test_ml_model.py::TestOutputValidation::test_tc_ml_10_return_value_contains_required_keys PASSED [ 83%]
tests/unit/test_ml_model.py::TestOutputValidation::test_tc_ml_11_urgency_level_is_valid PASSED [ 91%]
tests/unit/test_ml_model.py::TestOutputValidation::test_tc_ml_12_confidence_score_is_valid_float PASSED [100%]

===== 12 passed in 2.50s =====
```

Fig 2.6 - pytest_test_ml_model_results.png — 12 passed in 2.50s

2.7 Sprint 5A.6: Pipeline Integration Tests (test_pipeline.py — 6 tests)

The first integration tests exercise the full 3-layer pipeline — OpenAI extraction → CTAS rules → Random Forest — with all OpenAI calls mocked. No real API calls are made; tests run offline and produce deterministic results.

One important discovery: the correct patch target for `extract_symptoms()` is `components.chatbot.extract_symptoms`, not `components.openai_utils.extract_symptoms`. When `chatbot.py` does from `components.openai_utils` import `extract_symptoms`, it creates a local reference — patching the original module has no effect. The rule is: patch where the name is used, not where it is defined.

TC-PIPE-04 (emergency always wins) asserts both that status is "emergency" and that the keys `predicted_disease` and `confidence` are absent — proving the ML layer never ran.

```
(venv) PS C:\Users\user\OneDrive\Desmond_New\BC-Health-Platform> python -m pytest tests/integration/test_pipeline.py -v
===== test session starts =====
platform win32 -- Python 3.13.5, pytest-9.0.2, pluggy-1.6.0 -- C:\Users\user\OneDrive\Desmond_New\BC-Health-Platform\venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\user\OneDrive\Desmond_New\BC-Health-Platform
plugins: anyio-4.12.1
collected 6 items

tests/integration/test_pipeline.py::TestAssessmentPipeline::test_tc_pipe_01_emergency_circuit_breaker PASSED [ 16%]
tests/integration/test_pipeline.py::TestAssessmentPipeline::test_tc_pipe_02_ml_prediction_path PASSED [ 33%]
tests/integration/test_pipeline.py::TestAssessmentPipeline::test_tc_pipe_03_openai_fallback_path PASSED [ 50%]
tests/integration/test_pipeline.py::TestAssessmentPipeline::test_tc_pipe_04_emergency_always_wins PASSED [ 66%]
tests/integration/test_pipeline.py::TestAssessmentPipeline::test_tc_pipe_05_recommendation_never_empty PASSED [ 83%]
tests/integration/test_pipeline.py::TestAssessmentPipeline::test_tc_pipe_06_status_always_valid PASSED [100%]

===== 6 passed in 3.92s =====
```

Fig 2.7 - pytest_test_pipeline_results.png — 6 passed in 3.92s

2.8 Sprint 5A.7: Clinical Scenario Integration Tests (test_chatbot_scenarios.py — 6 tests)

Six end-to-end clinical scenarios tested through the full pipeline: cardiac emergency, stroke emergency, UTI ML prediction, vague symptom OpenAI fallback, emergency overriding mixed symptoms, and a single symptom edge case.

One correction during this sprint: UTI mock symptoms use space format ("burning micturition") not underscore format ("burning_urination") — because the feature vector builder normalizes spaces to underscores internally. The mock should reflect what OpenAI would realistically return before normalization.

```

(venv) PS C:\Users\user\OneDrive\Desmond_New\BC-Health-Platform> python -m pytest tests/integration/test_chatbot_scenarios.py -v
===== test session starts =====
platform win32 -- Python 3.13.5, pytest-9.0.2, pluggy-1.6.0 -- C:\Users\user\OneDrive\Desmond_New\BC-Health-Platform\venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\user\OneDrive\Desmond_New\BC-Health-Platform
plugins: anyio-4.12.1
collected 6 items

tests/integration/test_chatbot_scenarios.py::TestChatbotScenarios::test_tc_scen_01_cardiac_emergency PASSED [ 16%]
tests/integration/test_chatbot_scenarios.py::TestChatbotScenarios::test_tc_scen_02_stroke_emergency PASSED [ 33%]
tests/integration/test_chatbot_scenarios.py::TestChatbotScenarios::test_tc_scen_03_ml_prediction_util PASSED [ 50%]
tests/integration/test_chatbot_scenarios.py::TestChatbotScenarios::test_tc_scen_04_openai_fallback_vague PASSED [ 66%]
tests/integration/test_chatbot_scenarios.py::TestChatbotScenarios::test_tc_scen_05_emergency_overrides_ml PASSED [ 83%]
]
tests/integration/test_chatbot_scenarios.py::TestChatbotScenarios::test_tc_scen_06_single_non_emergency_symptom PASSED [100%]

===== 6 passed in 3.74s =====

```

Fig 2.8 - pytest_test_chatbot_scenarios_results.png — 6 passed in 3.74s

2.9 Sprint 5A.8: Dashboard Headless UI Tests (test_dashboard_apptest.py — 8 tests)

8 tests for the Personal Health Dashboard using Streamlit's AppTest framework — no browser required. Two discoveries during implementation: the dashboard's 4 stat cards are custom HTML via `st.markdown` (not `st.metric`), so tests assert on header text in the markdown output rather than metric components. The dashboard also uses a relative path for assets/logo.png that only resolves when the working directory is `Implementation/src/` — a `_run_dashboard()` helper handles this with `os.chdir()` in a `try/finally` block.

TC-UI-DASH-08 verifies data isolation by inspecting `mock_get.call_args` to confirm `get_user_assessments()` was called with the correct `user_id` from `session_state`.

```

(venv) PS C:\Users\user\OneDrive\Desmond_New\BC-Health-Platform> python -m pytest tests/ui/test_dashboard_apptest.py -v
===== test session starts =====
platform win32 -- Python 3.13.5, pytest-9.0.2, pluggy-1.6.0 -- C:\Users\user\OneDrive\Desmond_New\BC-Health-Platform\venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\user\OneDrive\Desmond_New\BC-Health-Platform
plugins: anyio-4.12.1
collected 8 items

tests/ui/test_dashboard_apptest.py::TestDashboardAppTest::test_tc_ui_dash_01_page_loads_without_exception PASSED [ 12%]
]
tests/ui/test_dashboard_apptest.py::TestDashboardAppTest::test_tc_ui_dash_02_page_renders_title PASSED [ 25%]
tests/ui/test_dashboard_apptest.py::TestDashboardAppTest::test_tc_ui_dash_03_stat_cards_present PASSED [ 37%]
tests/ui/test_dashboard_apptest.py::TestDashboardAppTest::test_tc_ui_dash_04_date_range_filter_present PASSED [ 50%]
tests/ui/test_dashboard_apptest.py::TestDashboardAppTest::test_tc_ui_dash_05_charts_render PASSED [ 62%]
tests/ui/test_dashboard_apptest.py::TestDashboardAppTest::test_tc_ui_dash_06_empty_state_no_crash PASSED [ 75%]
tests/ui/test_dashboard_apptest.py::TestDashboardAppTest::test_tc_ui_dash_07_unauthenticated_redirect PASSED [ 87%]
tests/ui/test_dashboard_apptest.py::TestDashboardAppTest::test_tc_ui_dash_08_user_data_isolation PASSED [100%]

===== 8 passed in 3.76s =====

```

Fig 2.9 - pytest_test_dashboard_apptest_results.png — 8 passed in 3.76s

2.10 Sprint 5A.9: Chatbot Headless UI Tests (test_chatbot_apptest.py — 8 tests)

8 tests for the chatbot page. Chat interaction is more complex than the dashboard because submitting a message triggers a full page rerun. A `_submit_chat()` helper was implemented that re-applies all mocks during the rerun — without this, AppTest would re-execute the page script and fire real OpenAI API calls.

The correct function to mock is `process_multiturn_message()` (not `run_assessment()`), since that is what the chatbot page calls directly for multi-turn dialog handling.

```
(venv) PS C:\Users\user\OneDrive\Desmond_New\BC-Health-Platform> python -m pytest tests/ui/test_chatbot_apptest.py -v
===== test session starts =====
platform win32 -- Python 3.13.5, pytest-9.0.2, pluggy-1.6.0 -- C:\Users\user\OneDrive\Desmond_New\BC-Health-Platform\venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\user\OneDrive\Desmond_New\BC-Health-Platform
plugins: anyio-4.12.1
collected 8 items

tests/ui/test_chatbot_apptest.py::TestChatbotAppTest::test_tc_ui_chat_01_page_loads_without_exception PASSED [ 12%]
tests/ui/test_chatbot_apptest.py::TestChatbotAppTest::test_tc_ui_chat_02_emergency_warning_banner PASSED [ 25%]
tests/ui/test_chatbot_apptest.py::TestChatbotAppTest::test_tc_ui_chat_03_welcome_message_shown PASSED [ 37%]
tests/ui/test_chatbot_apptest.py::TestChatbotAppTest::test_tc_ui_chat_04_submit_chat_no_crash PASSED [ 50%]
tests/ui/test_chatbot_apptest.py::TestChatbotAppTest::test_tc_ui_chat_05_ml_prediction_response PASSED [ 62%]
tests/ui/test_chatbot_apptest.py::TestChatbotAppTest::test_tc_ui_chat_06_emergency_response PASSED [ 75%]
tests/ui/test_chatbot_apptest.py::TestChatbotAppTest::test_tc_ui_chat_07_unauthenticated_blocked PASSED [ 87%]
tests/ui/test_chatbot_apptest.py::TestChatbotAppTest::test_tc_ui_chat_08_chat_history_persists PASSED [100%]

===== 8 passed in 3.55s =====
```

Fig 2.10 - pytest_test_chatbot_apptest_results.png — 8 passed in 3.55s

2.11 Sprint 5A.10: Property-Based Fuzz Tests (test_fuzz_hypothesis.py — 10 tests)

Property-based testing generates hundreds of random inputs automatically rather than relying on manually written test cases. With `max_examples=100` per test, 1,000 total random inputs were processed across 10 tests covering password validation, email validation, CTAS rules, and database operations.

TC-FUZZ-08 is the most healthcare-relevant test: it generates strings guaranteed not to match any CTAS keyword (using the `xyzzzy` prefix pattern) and asserts `is_critical=False` for all 100 examples — proving the emergency circuit breaker cannot be triggered by garbage or malicious input.

One correction during development: TC-FUZZ-02's password strategy was updated to require one letter plus one digit (any case), not uppercase plus lowercase separately — confirmed by reading `validate_password_strength()` in `auth.py`.


```

(venv) PS C:\Users\user\OneDrive\Desmond_New\BC-Health-Platform> python -m pytest tests/property/test_fuzz_hypothesis.py -v
===== test session starts =====
platform win32 -- Python 3.13.5, pytest-9.0.2, pluggy-1.6.0 -- C:\Users\user\OneDrive\Desmond_New\BC-Health-Platform\venv\Scripts\python.exe
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: C:\Users\user\OneDrive\Desmond_New\BC-Health-Platform
plugins: anyio-4.12.1, hypothesis-6.151.9
collected 10 items

tests/property/test_fuzz_hypothesis.py::TestPasswordFuzzing::test_tc_fuzz_01_never_crashes PASSED [ 10%]
tests/property/test_fuzz_hypothesis.py::TestPasswordFuzzing::test_tc_fuzz_02_valid_passwords_accepted PASSED [ 20%]
tests/property/test_fuzz_hypothesis.py::TestPasswordFuzzing::test_tc_fuzz_03_short_passwords_rejected PASSED [ 30%]
tests/property/test_fuzz_hypothesis.py::TestEmailFuzzing::test_tc_fuzz_04_never_crashes PASSED [ 40%]
tests/property/test_fuzz_hypothesis.py::TestEmailFuzzing::test_tc_fuzz_05_no_at_always_rejected PASSED [ 50%]
tests/property/test_fuzz_hypothesis.py::TestEmailFuzzing::test_tc_fuzz_06_valid_emails_accepted PASSED [ 60%]
tests/property/test_fuzz_hypothesis.py::TestCTASFuzzing::test_tc_fuzz_07_never_crashes PASSED [ 70%]
tests/property/test_fuzz_hypothesis.py::TestCTASFuzzing::test_tc_fuzz_08_garbage_never_triggers_emergency PASSED [ 80%]
]
tests/property/test_fuzz_hypothesis.py::TestCTASFuzzing::test_tc_fuzz_09_return_structure_consistent PASSED [ 90%]
tests/property/test_fuzz_hypothesis.py::TestDatabaseFuzzing::test_tc_fuzz_10_create_user_never_corrupts PASSED [100%]

===== 10 passed in 7.57s =====

```

Fig 2.10 - pytest_test_fuzz_hypothesis_results.png — 10 passed in 7.57s

2.12 Sprint 5A.11: Full Test Suite Run

The final Sprint 5A task ran all non-UI test files together to confirm the complete suite passes as one integrated pipeline. The command `python -m pytest tests/unit/ tests/integration/ tests/property/ -v` collected 79 test items and all passed in 20.89s. The UI tests (16 tests across `test_dashboard_apptest.py` and `test_chatbot_apptest.py`) were confirmed passing individually and are run from the `Implementation/src/` working directory. The complete test suite summary is shown in the table below.

Test File	Test Type	Tests Added	Cumulative Total	Status
tests/unit/test_auth.py	Unit	10	10	All Passed
tests/unit/test_ctas_rules.py	Unit	23	33	All Passed
tests/unit/test_database.py	Unit	12	45	All Passed
tests/unit/test_ml_model.py	Unit	12	57	All Passed
tests/integration/test_pipeline.py	Integration	6	63	All Passed

Test File	Test Type	Tests Added	Cumulative Total	Status
tests/integration/test_chatbot_scenarios.py	Integration	6	69	All Passed
tests/ui/test_dashboard_apptest.py	UI (AppTest)	8	77	All Passed
tests/ui/test_chatbot_apptest.py	UI (AppTest)	8	85	All Passed
tests/property/test_fuzz_hypothesis.py	Property (Fuzz)	10	95	All Passed
TOTAL	4 types	95	95	All Passing

2.13 Issues Encountered and Solutions

Issue	Impact	Solution
Old ad-hoc test scripts (test_openai.py, test_predict.py, test_rules.py) in Implementation/tests/ conflicted with new pytest suite	Running pytest tests/ from project root collected old scripts causing import errors and test confusion	Deleted the three old ad-hoc scripts from Implementation/tests/ — they were informal development verification scripts from Sprints 2A–2C, now superseded by the formal pytest suite
ML layer is embedded in chatbot.py, not a standalone ml_model.py as originally planned	test_ml_model.py could not import chatbot.py directly due to @st.cache_resource decorator and top-level Streamlit imports	Loaded model.pkl and model_metadata.json directly via joblib and json, replicating the feature vector building logic in the test file without importing chatbot.py or Streamlit
database.py functions call get_connection() internally	Standard conftest.py test_db fixture connection could not be	Used unittest.mock.patch to redirect get_connection() to a factory function that returns a connection to the temporary

Issue	Impact	Solution
— no connection parameter accepted	passed to database functions directly, risking writes to real health_platform.db	test database — same patch pattern used consistently across all database tests
Streamlit AppTest raises exception or stops early when st.switch_page() called for unauthenticated users	TC-UI-DASH-07 and TC-UI-CHAT-07 could not assert on a redirect that AppTest cannot follow	Used dual-path assertion: if at.exception is raised, auth guard fired correctly; else verified dashboard content (selectboxes, chat input) did not render — both cases prove the unauthenticated user was blocked
st.image('assets/logo.png') uses relative path that only resolves when CWD is Implementation/src/	AppTest ran from project root caused FileNotFoundError on logo asset, crashing dashboard and chatbot UI tests	Implemented os.chdir(SRC_DIR) with a try/finally block in all AppTest helper functions (_run_dashboard, _run_chatbot) to temporarily change working directory to Implementation/src/ during each test run
Wrong patch target for extract_symptoms() in integration tests — patched components.openai_utils.extract_symptoms instead of components.chatbot.extract_symptoms	Mock did not intercept the function call — the real OpenAI API was called during test execution	Corrected patch target to components.chatbot.extract_symptoms — patch where the name is looked up (chatbot module's namespace after from components.openai_utils import), not where it is defined
Hypothesis disallows @given tests with function-scoped pytest fixtures (test_db) by default	TC-FUZZ-10 (database fuzzing) could not use conftest.py test_db fixture —	Used pytest's tmp_path with a unique UUID-named SQLite file per generated example, then suppressed the health check with suppress_health_check=[HealthCheck.fun

Issue	Impact	Solution
	HealthCheck.function_scoped_fixture violation	ction_scoped_fixture] — each fuzz input gets its own isolated database

3. Repo Check-in of Implementation Completed

GitHub Repository: https://github.com/desporamon/W26_4495_S2_DesmondC

The following files were created since Progress Report 4. All work is committed with descriptive granular messages and pushed to the main branch. Each Sprint 5A sub-task has a corresponding commit.

File / Folder	Description
tests/conftest.py	New file: Shared pytest fixtures — test_db (in-memory SQLite via :memory:, yield teardown, row_factory=sqlite3.Row) and test_user (standard test user dict). Used by all unit tests that require database operations.
tests/TEST_REGISTER.md	New file: Human-readable test case register with table header (TC# Test Name Type Module Description Tool Status) — serves as the progress report documentation reference for all test cases
tests/unit/__init__.py, tests/integration/__init__.py, tests/ui/__init__.py, tests/property/__init__.py	New files: Empty Python package markers for all four test sub-folders
tests/unit/test_auth.py	New file: 10 unit tests for components/auth.py — registration (valid, duplicate, weak password, invalid email), login (valid, wrong password, non-existent email), security (bcrypt hash verification), session (logout clears state, page protection redirects)
tests/unit/test_ctas_rules.py	New file: 23 unit tests for components/rules.py — 14 parametrized positive detection tests (one per CTAS rule, dynamically built from

File / Folder	Description
	CRITICAL_RULES), 5 negative pass-through tests (non-emergency symptoms), 4 edge case tests (mixed case, None input, empty list, large lists)
tests/unit/test_database.py	New file: 12 unit tests for components/database.py — 5 User CRUD tests (create, get by email, non-existent email, duplicate email, bcrypt hash storage), 5 Assessment CRUD tests (save, retrieve by user, empty list, full field storage, invalid user_id), 2 Data Integrity tests (user isolation, table initialisation)
tests/unit/test_ml_model.py	New file: 12 unit tests for ML layer in chatbot.py — loads real model.pkl and model_metadata.json directly via joblib/json without importing Streamlit. Tests model loading (2), 70% confidence threshold logic (3), input handling (4), output validation (3)
tests/integration/test_pipeline.py	New file: 6 integration tests for full 3-layer assessment pipeline (OpenAI NLP → CTAS rules → Random Forest). Mocks extract_symptoms() and get_openai_fallback_guidance() — no real API calls. Tests emergency circuit breaker, ML prediction path, OpenAI fallback path, safety invariants
tests/integration/test_chatbot_scenarios.py	New file: 6 integration tests for 6 clinical scenarios — cardiac emergency (CTAS Level 1), stroke emergency (CTAS Level 1), UTI ML prediction (100% confidence), vague symptom OpenAI

File / Folder	Description
	fallback, emergency always overrides ML, minimal input edge case. All 6 passing on first run
tests/ui/test_dashboard_apptest.py	New file: 8 headless UI tests for 3_My_Dashboard.py using Streamlit AppTest — page load, title rendering, 4 stat card headers, 2 selectbox filters with correct options (All Time, Last 3 Months, All Urgency Levels), chart rendering, empty state, auth guard, user data isolation (user_id verified via mock call args)
tests/ui/test_chatbot_apptest.py	New file: 8 headless UI tests for 2_Symptom_Assessment.py — page load, medical disclaimer warning banner, personalised welcome message, chat input submission, ML prediction response, emergency response (CALL 911 content), auth guard, chat history persistence in session_state
tests/property/test_fuzz_hypothesis.py	New file: 10 property-based fuzz tests using Hypothesis (6.151.9) — 1,000 total random inputs across password validation (3 tests), email validation (3 tests), CTAS rules (3 tests), database create_user (1 test with UUID-named temp DB per example). TC-FUZZ-08 confirms garbage input never triggers false emergency

Git Commit History (this reporting period)

- Mar 21: "Sprint 5A.1: Remove old ad-hoc test scripts from Implementation/tests/ — superseded by proper pytest suite at root tests/"
- Mar 21: "Sprint 5A.3: Write test_ctas_rules.py — 23 unit tests covering all 14 CTAS rules, 5 negative, 4 edge cases — all passing"

- Mar 23: "Sprint 5A.4: Write test_database.py — 12 unit tests for user CRUD, assessment CRUD, data integrity — all passing"
- Mar 23: "Sprint 5A.5: Write test_ml_model.py — 12 unit tests for model loading, confidence threshold, input handling, output validation — all passing"
- Mar 23: "Sprint 5A.6: Write test_pipeline.py — 6 integration tests for full 3-layer pipeline (emergency, ML prediction, OpenAI fallback, safety invariants) — all passing"
- Mar 23: "Sprint 5A.7: Write test_chatbot_scenarios.py — 6 integration tests for clinical scenarios (cardiac, stroke, UTI, fallback, safety override, edge case) — all passing"
- Mar 23: "Sprint 5A.8: Write test_dashboard_apptest.py — 8 headless UI tests for dashboard page (load, title, stat cards, filters, charts, empty state, auth guard, data isolation) — all passing"
- Mar 23: "Sprint 5A.9: Write test_chatbot_apptest.py — 8 headless UI tests for chatbot page — all passing"
- Mar 23: "Sprint 5A.10: Write test_fuzz_hypothesis.py — 10 property-based fuzz tests, 1000 total inputs — all passing"

4. Revised Timeline and Next Steps

Sprint	Description	Dates	Status	Report
Sprint 0	Environment Setup	Jan 30–31	DONE	PR1
Sprint 1A–E	Foundation (Mockups, DB, Auth, Nav)	Feb 2–5	DONE	PR1
Sprint 2A–D	Symptom Assessment Chatbot (3-Layer Pipeline)	Feb 6–13	DONE	Midterm
Sprint 3A	Home Dashboard (UI rebuild)	Feb 25–27	DONE	PR2
Sprint 3B	Personal Health Dashboard + Enhancements	Feb 28 – Mar 8	DONE	PR2/PR3
Sprint 3E	Multi-Turn Chatbot Enhancement	Mar 8–9	DONE	PR3
Sprint 3C	BC Healthcare Facility Finder	Mar 9	DONE	PR3
Sprint 3D	Health Education Library	Mar 11–15	DONE	PR4
Sprint 4A	Synthetic Data Generation (5,000 records)	Mar 16	DONE	PR4
Sprint 4B	System Analytics Dashboard + K-Means	Mar 16	DONE	PR4
Sprint 5A	Automated Test Suite (Developer Testing)	Mar 17–21	DONE	PR5
Sprint 5B	GitHub Actions CI/CD Pipeline Setup	Mar 22–23	In Progress	PR5
Sprint 5C	Deployment + Smoke Check	Mar 23–24	Planned	PR5
Sprint 6	Documentation + Final Submission	Mar 25 – Apr 14	Planned	Final

4.1 What's Next: Sprint 5B, 5C and Sprint 6

Sprint 5B — GitHub Actions CI/CD (Mar 24–25): Create `.github/workflows/test.yml` to automatically run the full pytest suite on every git push to main. Every commit is validated before deployment. Sprint 5B also finalises `TEST_REGISTER.md` with all 95 test cases documented in plain English.

Sprint 5C — Deployment and Smoke Check (Mar 26–27): Push final code to GitHub, confirm CI passes, then Streamlit Cloud auto-deploys from the main branch — no manual step required, typically within 60 seconds. Configure `OPENAI_API_KEY` in Streamlit Cloud secrets and smoke check all 6 pages on the live URL.

Sprint 6 — Final Report and Submission (Mar 28 – Apr 14): Write the Final Report covering system architecture, design decisions, dual ML methodology, testing strategy, and CI/CD pipeline. Prepare the final presentation and update the GitHub README with deployment instructions and how to run the test suite locally.

4.2 Scope Decision: Community Pulse Formally Descoped

The Community Pulse (Reddit scraping) feature that was mentioned in Progress Report 4 has been officially dropped. The Development time was shifted to Sprint 5A's automated testing suite which makes the academic contribution stronger for the platform that is already feature complete. The feature will be mentioned in the Final Report under Future Work.

4.3 QA Perspective Testing — Post-Submission Portfolio

As a student applying for QA roles, a post-submission extension is planned to add Playwright end-to-end regression tests against the live Streamlit Cloud URL — complementing the developer-side pytest suite with real browser testing. This mirrors industry practice: developer tests (pytest + AppTest) gate deployment via CI, while QA regression tests (Playwright) validate the live environment after deployment. This will be completed post-submission as a portfolio piece.

5. AI Use Section

5.1 AI Tools Usage Summary

In this reporting period, AI tools helped me understand technical concepts and patterns specific to the framework before I implemented them. All testing architecture decisions — the four-folder test structure, the `conftest.py` fixture design, which functions to mock in each test file, the clinical scenarios for integration tests, the Hypothesis strategies for fuzz tests — were made independently based on reading the actual source code of each module being tested. AI tools were used in the same way as a developer might consult the framework documentation or a technical colleague. All test results were verified by running `pytest` independently in the terminal and reassuringly confirming pass/fail results.

5.2 Table of AI Tools and Specific Use

AI Tool	Version, Account Type	Specific Use	Value I Added
Claude (Anthropic)	Claude.ai, Free/Pro account	Understanding <code>pytest</code> fixture patterns and <code>conftest.py</code> design — how the <code>test_db</code> fixture with <code>yield teardown</code> and <code>:memory:</code> SQLite provides isolated test databases without touching production data	Independently decided on the four-folder test architecture (unit, integration, ui, property) based on the project's testing requirements. Designed the fixture structure to match the database module's actual connection pattern. All fixture parameter choices (<code>row_factory</code> , <code>PRAGMA foreign_keys</code>) were determined by

AI Tool	Version, Account Type	Specific Use	Value I Added
			reading the real source code
Claude (Anthropic)	Claude.ai, Free/Pro account	Understanding how to mock Streamlit dependencies (st.cache_resource, st.session_state, st.secrets) so that chatbot.py could be imported in test environments without a running Streamlit server	Identified which Streamlit calls needed mocking by reading chatbot.py line by line. Designed the sys.modules patching approach that installs mock Streamlit before import. All mock return values were constructed to match the real function signatures I verified from source code
Claude (Anthropic)	Claude.ai, Free/Pro account	Understanding property-based testing concepts with Hypothesis — @given decorator, strategies (st.text, st.lists, st.from_regex), @settings for controlling example count, and the HealthCheck.function_scoped_fixture suppression pattern	Independently designed all 10 fuzz test scenarios based on security and safety requirements of the healthcare platform. Identified the _CTAS_KEYWORDS set approach for TC-FUZZ-08 (garbage never triggers emergency) — ensuring non-medical gibberish uses the

AI Tool	Version, Account Type	Specific Use	Value I Added
			xyzzzy prefix pattern that cannot match any real CTAS keyword. The 1,000 total fuzz inputs was my design decision
Claude (Anthropic)	Claude.ai, Free/Pro account	Understanding Streamlit AppTest framework — <code>AppTest.from_file()</code> , <code>session_state</code> injection, <code>at.chat_input[0].set_value().run()</code> syntax for simulating chat submission, and how AppTest handles <code>st.switch_page()</code> for auth guard testing	Identified the <code>os.chdir(SRC_DIR)</code> requirement by running the tests and diagnosing the <code>FileNotFoundError</code> on <code>assets/logo.png</code> myself. Designed the <code>_run_dashboard()</code> and <code>_run_chatbot()</code> helper pattern for test reuse. Determined which components to assert (selectbox count, markdown content, warning count) based on reading the actual page source code
Claude Code (VS Code)	Extension, integrated with Claude	Generating test file boilerplate from my detailed specifications — implementing each test case after I described the exact function to test,	All test case design decisions were mine: which 10 auth scenarios to cover, which 14 CTAS rules to parametrize, which 12

AI Tool	Version, Account Type	Specific Use	Value I Added
		the expected behaviour, and the assert conditions to verify	database operations to validate, which 6 clinical scenarios to test end-to-end. I read the source code for each module before writing prompts, and verified all outputs by running tests myself and checking pass/fail results

5.3 Statement of Originality

All Sprint 5A testing architecture decisions were independently made — the four-folder structure, conftest.py fixture design, parametrize approach for CTAS rules, patching at the point of use, os.chdir() for AppTest asset resolution, and UUID-named databases for Hypothesis fuzzing. These were arrived at by running tests, observing failures, and diagnosing root causes.

AI tools were used to understand framework mechanics (pytest fixtures, Hypothesis strategies, AppTest), not to design test cases. The 95 test cases reflect my independent judgement of what a healthcare AI application needs verified: bcrypt security, CTAS rule coverage, user data isolation, ML confidence thresholds, and resistance to garbage input. Claude Code generated boilerplate from my detailed specifications — all test case IDs, assertion conditions, and mock structures were defined by me after reading each source file.

Appendix A: AI Prompt History

The following documents AI tool interactions during this reporting period. AI was used to clarify technical concepts and understand framework-specific patterns. Each entry follows the Context / Prompt / Summary / Value I Added structure.

Prompt Session 1: Understanding pytest Fixture Patterns for conftest.py Design

Date: March 17, 2026 | **Tool:** Claude (Anthropic)

Context: Having decided on the four-folder test structure, I needed to understand how conftest.py fixtures work — specifically the yield teardown pattern and how :memory: SQLite ensures tests never touch the real health_platform.db. I also needed to understand how pytest discovers and applies fixtures automatically to test functions that list them as parameters.

My Prompt: *"I'm setting up a pytest test suite for a Streamlit + SQLite healthcare application. I need a conftest.py with a test_db fixture that creates a temporary in-memory SQLite database for each test and tears it down afterwards. How does the yield keyword work in pytest fixtures for setup and teardown? And what does sqlite3.connect(':memory:') do exactly — is it truly isolated per test?"*

Summary of Response: Claude explained the yield-based fixture pattern where code before yield runs as setup and code after yield runs as teardown after the test completes. It clarified that :memory: creates a database entirely in RAM that is automatically destroyed when the connection is closed, ensuring complete isolation. It also explained that by default pytest fixtures are function-scoped — a new instance is created for each test function.

Value I Added: I independently decided on both fixtures in conftest.py (test_db and test_user) based on reviewing which tests would need database access versus which only needed authentication data. I added row_factory = sqlite3.Row and PRAGMA foreign_keys = ON to the test_db fixture by reading the real get_connection() implementation in database.py — these were not suggested by Claude. The decision to mirror the real connection setup exactly was mine, ensuring test conditions match production behaviour.

Prompt Session 2: Mocking Streamlit Dependencies for Chatbot Module Testing

Date: March 19, 2026 | **Tool:** Claude (Anthropic)

Context: After discovering the ML layer lives in chatbot.py and that chatbot.py uses @st.cache_resource on load_model() and imports streamlit at the top level, I needed to understand how to import

chatbot.py in a test environment where Streamlit is not running — without triggering st.secrets access, @st.cache_resource decoration failures, or st.session_state errors.

My Prompt: *"I need to import a Python module (chatbot.py) that has 'import streamlit as st' at the top level and uses @st.cache_resource as a decorator on a function. When I try to import it in pytest, it fails because streamlit tries to connect to a server. How can I mock the entire streamlit module in sys.modules before importing chatbot.py so that all st.* calls are intercepted by MagicMock?"*

Summary of Response: Claude explained the sys.modules patching approach — installing a MagicMock as sys.modules['streamlit'] before any import of the module that depends on it. It noted that @st.cache_resource can be replaced with a passthrough lambda that returns the decorated function unchanged, and that st.secrets can be mocked as a plain dict.

Value I Added: I identified which specific Streamlit attributes needed specific mock values by reading chatbot.py line by line — st.secrets needed {"OPENAI_API_KEY": "test-key"} because openai_utils.py accesses it at import time on line 35. The cache_resource lambda pattern (lambda func=None, **kw: func if func else (lambda f: f)) was verified by testing against the actual decorator usage in chatbot.py. All mock return value structures were constructed from reading the real function signatures.

Prompt Session 3: Property-Based Testing with Hypothesis for Healthcare Validation Functions

Date: March 21, 2026 | **Tool:** Claude (Anthropic)

Context: Having designed the 10 fuzz test scenarios, I needed to understand how to compose Hypothesis strategies for inputs that satisfy specific structural constraints — particularly valid password generation (8+ chars, letter + digit but NOT requiring uppercase + lowercase separately as I had initially assumed) and valid email generation (local@domain.tld satisfying all 6 checks in validate_email()).

My Prompt: *"I'm writing property-based tests with Hypothesis for a validate_email() function that checks: non-empty after strip, exactly one '@', non-empty local part, domain contains '.', domain doesn't start/end with '.', TLD is 2+ chars. How do I build a Hypothesis strategy using st.builds() that generates emails guaranteed to satisfy all 6 conditions? Also, for TC-FUZZ-08 where I need strings that never match any of 40+ medical keywords, what is the most reliable approach?"*

Summary of Response: Claude explained the st.builds() strategy for composing structured strings from independently generated parts, and the st.from_regex() strategy for generating strings matching specific patterns. For the non-medical keyword filter, it suggested the st.from_regex() approach with a prefix like 'xyzyz' guaranteed not to appear in medical terminology.

Value I Added: The decision to use `@settings(max_examples=100)` for exactly 1,000 total fuzz inputs was mine — balancing coverage against test suite execution time. The `_CTAS_KEYWORDS` set containing all 40+ keywords across all 14 CTAS rules was compiled by me from reading `rules.py` directly. The `valid_password` strategy correction (letter + digit, not uppercase + lowercase) was caught by me reading `validate_password_strength()` source code and comparing to my initial prompt assumptions — I corrected this before writing the tests. The TC-FUZZ-10 UUID-named database file approach was my solution to the Hypothesis `function_scoped_fixture` health check issue.

Prompt Session 4: Streamlit AppTest Framework for Headless UI Testing

Date: March 20, 2026 | **Tool:** Claude (Anthropic)

Context: Before writing the dashboard and chatbot UI tests, I needed to understand Streamlit's AppTest API — specifically how to set `session_state` before running the page, how to simulate chat input submission, how AppTest handles `st.switch_page` for auth guard testing, and the limitations around Plotly chart rendering.

My Prompt: *"I'm using Streamlit's AppTest to test a dashboard page that requires authenticated session state. How do I set `st.session_state` values before calling `at.run()`? The page also has `st.chat_input` — what is the AppTest syntax to simulate a user submitting a chat message? And what happens in AppTest when `st.switch_page()` is called for unauthenticated users — does it raise an exception or just stop rendering?"*

Summary of Response: Claude explained the `AppTest.from_file()` pattern, `session_state` injection via `at.session_state["key"] = value` before `run()`, the `at.chat_input[0].set_value("message").run()` syntax for chat submission, and that `st.switch_page()` in AppTest context typically raises a `StreamlitAPIException` or causes early page execution termination.

Value I Added: The `os.chdir(SRC_DIR)` fix for the `assets/logo.png` relative path issue was discovered through my own testing — running the tests, diagnosing the `FileNotFoundError`, tracing it to the relative path in `st.image()`, and implementing the `try/finally` pattern. The dual-path assertion for TC-UI-DASH-07 and TC-UI-CHAT-07 (check both `at.exception` and absence of page content) was my design for handling both possible AppTest behaviours. The `_submit_chat()` helper with patch re-application was my solution for ensuring mocks persist across the page rerun triggered by chat input submission.